Design and Implementation of a Secure MQTT-Based IoT Communication System Using ECC, AES, HMAC, and Access Control Model

Submitted by:

Sidrah AL Ajlouny

Tareq AL shhadat

Under the supervision of :

DR. Wasim Al juniedi

ENG. Abdulrahman Arar

2026/2027

Abstract

Table of contents

Introduction.....

1.1 Introduction.....

1.2 Problem Statement and Motivation.....

1.3 Project Objectives.....

1.4 Previous Studies and Related Work.....

1.5 Practical Applications of the Project.....

1.6 Scientific Challenges.....

Chapter 1: Theoretical Framework and literature Review.....

1.1 Introduction to the Internet of Things (IoT) …….

1.2 IoT Architecture…….

1.3 MQTT Protocol……

1.4 MQTT Architecture and Communication Model……

1.5 IoT Security Challenges……

1.6 Common Attacks on MQTT-Based IoT Systems……

1.7 Cryptography in IoT……

1.8 Hash-Based Message Authentication Code (HMAC)……

1.9 Why HMAC Is Suitable for MQTT-Based IoT Systems……

1.10 Authorization and Access Control Models……

List of Figures

Figure 1.1: Overview of the Internet of Things (IoT)

Figure1. 2: Three-Layer Architecture of the Internet of Things

Figure 1.3: MQTT Protocol in IoT Communication

Figure 1.4: Common Security Challenges in IoT Systems

Figure 1.5: Common Security Threats in MQTT-Based IoT Systems

Figure 1.6: Role of Cryptography in IoT Security

Figure 1.7: HMAC-Based Message Authentication Process

Figure 1.8: Overview of Access Control Models

List of Tables

Table 1: *Article Table – Related Research Papers*
Table 2: *IBAC, RBAC and ABAC comparison*
Table 3:
Table 4:
Table 5:
Table 6:

List of Abbreviations

Abbreviation	Definition
IOT	Internet of Things
MQTT	Message queuing telemetry transport
AES	Advanced Encryption Standard
ECC	Elliptic curve cryptography
HMAC	Hash-Based Message Authentication Code
IBAC	Identity-Based Access Control
ABAC	Attribute-Based Access Control
RBAC	Role-Based Access Control
KDF	Key Derivation Function
MitM	Man-in-the-Middle
DoS	Denial of Service

List of Symbols

Symbol	Definition

Introduction

1.1 Introduction

The rapid growth of the Internet of Things (IoT) has significantly transformed modern information systems by enabling interconnected devices to collect, exchange, and process data across applications such as smart homes, healthcare, industrial automation, and environmental monitoring. However, this increasing connectivity introduces important cybersecurity challenges, including unauthorized access, data interception, message manipulation, device impersonation, and privacy violations. These risks are particularly significant in IoT environments, where devices often operate with limited computational resources and require efficient security mechanisms. The MQTT protocol is widely used for IoT communication because of its lightweight publish/subscribe architecture, which facilitates communication between devices through a central broker. Nevertheless, securing MQTT-based systems requires more than reliable message delivery; appropriate mechanisms are needed to protect data confidentiality and integrity while ensuring that only authorized devices can access specific resources [1], [2].

Cryptographic techniques provide essential foundations for addressing these challenges. The Advanced Encryption (AES) protects sensitive information against unauthorized disclosure, while Elliptic Curve Cryptography (ECC) supports secure key-establishment mechanisms with relatively small key sizes. In addition, Hash-Based Message Authentication Code (HMAC) enables message integrity verification and authentication based on possession of a shared secret key [3]–[5]. Although these mechanisms provide important security properties, encryption and message authentication do not independently define which devices should be permitted to publish data, subscribe to MQTT topics, or perform particular operations. Consequently, **authorization** is an essential complementary security mechanism that determines whether an authenticated device has sufficient permission to access a requested resource. The MQTT Version 5.0 specification explicitly discusses client authorization and access controls for restricting publication to specific topics and subscriptions to particular topic filters [1].

The specific objective of this project is to develop and evaluate a secure MQTT communication framework between two simulated IoT devices by combining ECC-based key establishment, AES encryption, and HMAC-based message verification with an authorization layer. The proposed extension considers three access control approaches: Identity-Based Access Control (IBAC), Role-Based Access Control (RBAC), and Attribute-Based Access Control (ABAC). IBAC associates permissions with individual device identities, RBAC assigns permissions according to predefined roles, and ABAC evaluates attributes of the device, requested operation, resource, and potentially the operating environment against defined policies [6], [7]. These approaches provide different levels of flexibility and administrative complexity for controlling access in IoT systems.

The importance of this project lies in integrating **cryptographic protection with explicit access control** rather than relying exclusively on message encryption and integrity verification. By evaluating authorized and unauthorized communication attempts, message-tampering scenarios, and the processing overhead introduced by different authorization models, the project aims to investigate how access control can complement existing cryptographic protections in a resource-conscious IoT environment. The resulting evaluation can help identify the practical trade-offs between security, implementation complexity, and communication latency, providing a foundation for extending the framework to larger IoT deployments.

1.2 Problem Statement and Motivation

The increasing adoption of the Internet of Things (IoT) has created a growing need for secure and reliable communication between interconnected devices. In IoT environments, devices continuously exchange sensitive information through lightweight communication protocols such as Message Queuing Telemetry Transport (MQTT). Although MQTT provides an efficient publish/subscribe communication model, the security of an MQTT-based system depends on the appropriate implementation of authentication, authorization, confidentiality, and integrity mechanisms. Without effective access control, unauthorized devices may attempt to publish messages, subscribe to sensitive topics, or access resources beyond their intended permissions. Furthermore, transmitted messages may be exposed to interception, modification, or impersonation attacks if adequate security mechanisms are not implemented [1], [2].

A major challenge in securing IoT communication is achieving an appropriate balance between security and computational efficiency. Traditional security mechanisms may introduce additional processing overhead, which can be significant for resource-constrained devices. Cryptographic techniques can address several aspects of this problem: the Advanced Encryption Standard (AES) provides data confidentiality, Elliptic Curve Cryptography (ECC) supports secure key establishment, and Hash-Based Message Authentication Code (HMAC) enables integrity verification and message authentication using a shared secret key [3]–[5]. However, these mechanisms do not independently establish a complete authorization policy. In particular, successful decryption or message verification does not necessarily determine whether a device should be permitted to publish to a particular MQTT topic or access a specific resource.

Therefore, the scientific problem addressed by this project is how to integrate cryptographic protection with an explicit authorization mechanism in an IoT communication environment while maintaining acceptable communication performance. The proposed solution extends an existing MQTT-based communication framework consisting of two simulated IoT devices. The framework combines ECC-based key establishment, AES encryption, and HMAC verification with an authorization layer that evaluates whether a device is permitted to perform a requested operation. Three authorization approaches are considered: Identity-Based Access Control (IBAC), Role-Based Access Control (RBAC), and Attribute-Based Access Control (ABAC). These approaches represent different ways of defining access permissions, ranging from device-specific identities to assigned roles and policies based on relevant attributes [6], [7].

The proposed framework will be evaluated under normal communication conditions and unauthorized-access scenarios. Tests will examine whether authorized devices can exchange protected messages, whether unauthorized publication or subscription attempts are rejected, and whether message modification is detected by the existing cryptographic mechanisms. The evaluation will also consider the additional processing delay introduced by authorization. By combining these security layers, the project aims to demonstrate how access control can complement cryptographic protection and improve the overall security of MQTT-based IoT communication without unnecessarily compromising efficiency.

1.3 Project Objectives

The main objective of this project is to develop and evaluate a secure communication framework for simulated Internet of Things (IoT) devices using MQTT, combining cryptographic protection with authorization mechanisms to restrict access to communication resources. The project builds upon an existing security architecture based on Elliptic Curve Cryptography (ECC), the Advanced Encryption Standard (AES), and Hash-Based Message Authentication Code (HMAC), extending it with access control policies that determine which devices are permitted to perform specific operations.

The specific objectives of the project are as follows:

1. **Implement secure MQTT communication:** Establish communication between two simulated IoT devices through an MQTT broker while preserving the existing publisher–subscriber architecture.

2. **Protect message confidentiality:** Use AES encryption to prevent unauthorized parties from reading transmitted IoT data.

3. **Establish cryptographic keys securely:** Apply ECC-based key establishment to derive a shared secret, which is used to derive the symmetric encryption key through an appropriate key derivation function.

4. **Verify message integrity and authenticity:** Use HMAC to detect unauthorized message modifications and verify message authenticity based on possession of the shared secret, while recognizing that HMAC alone does not establish a unique device identity.

5. **Integrate authorization into the communication framework:** Introduce an access control layer that evaluates device permissions before allowing MQTT publication, subscription, or other defined operations.

6. **Implement and examine three authorization approaches:** Investigate Identity-Based Access Control (IBAC), in which permissions are associated with registered device identities; Role-Based Access Control (RBAC), in which permissions are assigned through roles; and Attribute-Based Access Control (ABAC), in which access decisions are determined by evaluating relevant device, resource, operation, and environmental attributes against defined policies [6], [7].

7. **Detect and reject unauthorized access attempts:** Test scenarios involving unregistered devices, unauthorized MQTT topic access, and attempts to perform operations beyond assigned permissions. Record access decisions and relevant security events.

8. **Evaluate the framework's security and performance:** Measure message delivery time and the additional processing overhead associated with authorization. Compare the behavior of the authorization approaches under consistent experimental conditions, including their ability to reject unauthorized requests and permit legitimate communication.

9. **Assess the practical trade-offs between security and efficiency:** Examine the differences in policy flexibility, implementation complexity, and communication overhead to identify the advantages and limitations of each authorization approach in the simulated IoT environment.

Through these objectives, the project seeks to provide an experimental evaluation of the integration of authorization and cryptographic protection in MQTT-based IoT communication. The intended outcome is a modular security framework that preserves confidentiality and message integrity while enforcing explicit access permissions, together with measurable results that support comparisons between the selected authorization approaches.

1.4 Previous Studies and Related Work

Study 1 — Analysis and Implementation of Lightweight Key Exchange Algorithms for MQTT Security (2026)

Authors: Rahul Kumar Singh, Ashutosh Kumar, S. Venkatesan, Manish Kumar, and Sandeep Kumar Shukla. Publisher: Elsevier, Computer Networks.

Summary and relationship to the project:

This study investigates lightweight key-sharing methods for securing MQTT communications in resource-constrained Internet of Things environments. The authors implement and compare several key-sharing techniques using the Python-based mqtt framework. The investigated methods

include polynomial-based approaches, the Chinese Remainder Theorem, Walsh-code-based techniques, hash chains, and XOR-based mechanisms. The study also considers the lightweight ASCON algorithm and evaluates computational efficiency, scalability, and execution time in a simulated IoT environment. Its results identify a randomized-number-based XOR approach as a promising option in the evaluated setting.

The study is relevant to the proposed project because secure communication depends not only on the encryption algorithm but also on how cryptographic keys are established and distributed. Our project uses ECC-based key agreement and HKDF-SHA256 to derive an AES key. This paper provides a useful comparison point for discussing alternative lightweight key-management techniques and for justifying the need to evaluate key-establishment overhead. However, its key-sharing design is not identical to the ECC–HKDF method used in our project.

Study 2 — Efficient and Dynamic Access Control with End-to-End Message Security for MQTT (2026)

Authors: Liang Chen and James Wilson. Publisher: Elsevier, Array, Volume 30, Article 100972.

Summary and relationship to the project

This paper proposes an access-control policy model and an enforcement mechanism designed specifically for MQTT. A key contribution is the ability to assign and maintain access-control labels as new topics are introduced into a changing MQTT topic hierarchy. The authors combine policy enforcement with end-to-end payload protection using lightweight symmetric cryptography. This approach aims to prevent the broker and intermediate components from reading message plaintext, in contrast to relying solely on hop-by-hop protection such as MQTT over TLS. The paper also evaluates computational and memory overhead and reports advantages over TLS in its tested environment.

This study is particularly relevant because our project addresses two distinct security requirements: protecting the message content and determining which devices are permitted to publish or subscribe. The paper supports treating authorization as a separate security layer rather than assuming that encryption alone prevents unauthorized access. It also provides a useful basis for discussing MQTT topic policies and performance trade-offs. The proposed project can complement this line of work by examining IBAC, RBAC, and ABAC alongside ECC-based key establishment, AES encryption, and HMAC verification.

Study 3 — A Systematic Review of Lightweight Cryptographic Schemes for Security and Privacy in IoT (2025)

Authors: Shaharyar Alam Ansari and Salman Ali. Publisher: Springer Nature, Discover Computing, Volume 28, Article 266.

Summary and relationship to the project

This systematic literature review examines 77 peer-reviewed studies published between 2006 and 2025 on lightweight cryptographic approaches for IoT security and privacy. It categorizes techniques used to provide confidentiality, integrity, and authentication, and discusses the performance costs associated with cryptographic protection on devices with limited processing power, memory, and energy. The review also considers ECC optimization, hybrid cryptographic frameworks, and the integration of cryptography with emerging technologies. It identifies a continuing need for more comprehensive solutions and stronger practical evaluation on embedded hardware.

The paper provides a broad academic foundation for the cryptographic choices made in our project. ECC supports the establishment of shared key material, AES protects message confidentiality, and HMAC helps detect unauthorized message modification when the communicating parties securely share the relevant secret. The review is useful for explaining why combining complementary mechanisms can be more appropriate than relying on a single technique. It also supports evaluating

not only whether tampering is detected, but also the associated execution time and resource overhead. As a review, however, it does not itself implement our specific MQTT architecture or the same combination of algorithms.

Study 4 — A Novel MQTT-ZT Secure Broker: Zero Trust Architecture for IoT Security (2025)

Authors: Meha James, Thomas Newe, Donna O'Shea, and George D. O'Mahony. Publisher: IEEE, Proceedings of the 2025 IEEE International Conference on Cyber Security and Resilience (CSR), pp. 433–439.

Summary and relationship to the project

This paper introduces the MQTT-ZT Secure Broker, an MQTT-based architecture that applies Zero Trust principles to IoT communication. Its design integrates Attribute-Based Access Control (ABAC) into the broker through a Policy Enforcement Point (PEP) and an ALFA policy engine. Instead of relying only on a device's general role or initial access, the system evaluates authorization policies when clients perform communication operations. The reported experiments indicate that the broker maintains low authorization-processing times for publishers and subscribers in tests involving up to 200 clients.

The study is directly relevant to the authorization component of our project. It demonstrates how MQTT publish and subscribe operations can be governed by explicit policies, which complements our use of cryptography to protect messages and detect tampering. For our implementation, similar principles could be applied to permit a registered sensor to publish to an approved topic while preventing an unauthorized device from publishing or subscribing elsewhere. The principal difference is that this paper emphasizes Zero Trust and broker-side ABAC, whereas our project combines cryptographic message protection with an examination of IBAC, RBAC, and ABAC.

Study 5 — Enabling Robust Security in MQTT-Based IoT Networks with Dynamic Resource-Aware Key Sharing (2025)

Authors: Sharadadevi Kaganurmath and Nagaraj G. Cholli. Publisher: Elsevier, Procedia Computer Science, Volume 252, pp. 633–642.

Summary and relationship to the project

This paper proposes Dynamic Lightweight Authentication for MQTT (DLA-MQTT), a mechanism intended to improve the security of MQTT communication while accounting for the limited resources of IoT devices. The proposed method uses a Generalized Feedback Shift Register (GFSR)-based pseudo-random number generator to create ephemeral keys. The authors compare the approach with existing cryptographic techniques and evaluate it using the Cooja simulator under different threat scenarios. The study reports improvements in computational overhead, energy consumption, and execution time, and examines resistance to attacks such as Man-in-the-Middle (MitM) and Denial of Service (DoS).

The study is relevant to our project because both systems seek to secure MQTT communication without introducing excessive overhead. It offers a comparison point for the design of lightweight authentication and key-management mechanisms. Our project, however, follows a different cryptographic design: ECC-based key agreement, HKDF-SHA256 key derivation, AES-EAX encryption, and HMAC-SHA256 verification. This distinction should be made clear in the report. The paper can support the discussion of lightweight security and performance evaluation, but its results should not be presented as measurements of our own system.

Study 6 — Experimental and Data-Driven Evaluation of Transport-Layer and Application-Layer Security for MQTT-Based IoT Networks (2026)

Authors: Carolina Del-Valle-Soto, Maria Fernanda Alvarez-Garcia, Leonardo J. Valdivia, José A. Del-Puerto-Flores, José Varela-Aldás, and Paolo Visconti. Publisher: Elsevier, Pervasive and Mobile Computing. OBJ

Summary and relationship to the project:

This study compares two security approaches for MQTT-based IoT systems implemented on ESP32 devices. The first uses transport-layer security through TLS, while the second uses application-layer encryption with ECC for key exchange and AES for symmetric encryption. The researchers evaluate the approaches under comparable conditions by measuring memory use, end-to-end latency, energy consumption, and resistance to passive traffic interception. They also use multivariate statistical analysis to examine the performance characteristics of each configuration. The reported findings highlight a trade-off: TLS provides strong confidentiality protection but incurs greater memory use, while the ECC–AES approach reduces memory requirements with some latency cost.

This paper closely matches the cryptographic architecture of our project because both approaches use ECC for key establishment and AES for message confidentiality in MQTT environments. Its evaluation method is particularly useful when deciding which measurements to report, including message-delivery delay and resource overhead. Our project additionally uses AES-EAX authentication and HMAC-SHA256 verification, and examines access-control models. Therefore, the paper can inform our methodology and comparison discussion, but it does not establish the performance of our particular implementation.

Study 7 — Scalable and Energy Efficient Hybrid Cryptographic Framework for IoT Security Using Advanced Symmetric and Asymmetric Techniques (2026)

Authors: Bijie Huan and Haiyan Xie. Publisher: Springer Nature, Scientific Reports, Volume 16, Article 17710.

Summary and relationship to the project

This paper presents a Hybrid Elliptic-AES Encryption (HEAE) framework that combines AES-256 symmetric encryption with ECC-based key management. Its goal is to balance cryptographic protection with the resource constraints of IoT devices. The authors evaluate the framework through simulations and compare its execution time and energy consumption with other cryptographic approaches, including RSA and DES. The article reports improvements in selected execution-time and energy measurements and discusses possible integration with IoT technologies such as MQTT, Zigbee, and LoRaWAN.

The study is relevant because it follows the same general hybrid-cryptography principle used in our project: asymmetric cryptography establishes key material, while symmetric encryption protects the actual data. This division of responsibilities helps avoid using computationally expensive public-key operations for every message. Our project uses ECC with ECDH and HKDF-SHA256 to derive an AES key, then protects messages using AES-EAX and verifies HMAC-SHA256. The paper supports the rationale for a hybrid design, although its specific HEAE implementation and evaluation are not identical to ours.

Study 8 — XMID-MQTT: Explaining Machine Learning-Based Intrusion Detection System for MQTT Protocol in IoT Environment (2025)

Authors: Hayette Zeghida, Mehdi Boulaiche, Ramdane Chikh, Ahmed Patel, Ana Luiza Bessa Barros, and co-authors. Publisher: Springer Nature, International Journal of Information Security, Volume 24, Article 128.

Summary and relationship to the project

This study addresses intrusion detection in MQTT-based IoT networks using machine-learning methods, with an emphasis on making detection decisions more explainable. MQTT environments may be exposed to a range of attacks, and detecting suspicious traffic is important when protocol-

level protection alone is insufficient. The study explores an intrusion-detection approach that combines machine learning with explainability so that detected patterns can be interpreted rather than treated as opaque model outputs. This can help security analysts understand why a network event is considered suspicious.

The paper is relevant to the penetration-detection aspect of our project, but its approach differs from our cryptographic verification mechanism. Our current design detects a modified message by checking the AES-EAX authentication tag and recomputing HMAC-SHA256. XMID-MQTT instead focuses on identifying potentially malicious behavior in MQTT traffic using machine learning. It can therefore be used to distinguish cryptographic integrity verification from network-level intrusion detection in the literature review. These methods address related but different security problems and could be complementary in a future extension.

Study 9 — Enhancing MQTT Intrusion Detection in IoT Using Machine Learning and Feature Engineering (2025)

Authors: Iwan Handoyo Putro, Tohari Ahmad, and Royyana Muslim Ijtihadie. Publisher: IEEE, IEEE Open Journal of the Communications Society, Volume 6, pp. 7855–7884.

Summary and relationship to the project

This paper investigates how feature engineering and machine-learning algorithms can improve intrusion detection in MQTT-based IoT networks. The authors evaluate seven models: Stochastic Gradient Descent, Logistic Regression, Random Forest, Decision Tree, Naïve Bayes, k-Nearest Neighbors, and XGBoost. Their approach uses Euclidean-distance-based feature generation to improve the classification of abnormal network traffic. The study reports strong classification performance for selected models and shows that feature engineering can improve detection quality, including reducing false-positive rates in the tested dataset.

The connection to our project is the shared goal of identifying and rejecting malicious communication. However, our current approach is not a machine-learning IDS: it uses cryptographic checks to reject messages whose authenticated-encryption tag or HMAC verification fails. The paper is useful for explaining how a future version could extend the system to detect abnormal MQTT behavior, such as suspicious traffic patterns or flooding attacks that may not be identified merely by checking message integrity. It also provides examples of evaluation metrics such as precision, recall, F1-score, and false-positive rate, should an ML-based extension be implemented.

Study 10 — A Novel Risk-Based Access Control Engine in Zero Trust Architecture for IoT Network (2025)

Authors: Yi-Wei Ma and Po-Hung Chiu. Publisher: Springer Nature, International Journal of Information Security, Volume 24, Article 124.

Summary and relationship to the project

This study proposes a Risk-Based Access Control Engine (RBACE) within a Zero Trust architecture for IoT networks. Rather than treating a device as trustworthy simply because it has connected to the network, the framework continuously evaluates node behavior and risk. Its decision process considers factors such as node capabilities, observed activities, communication features, and data content. The authors evaluate the approach against four attacks involving the RPL routing protocol and report that it can detect and mitigate these attacks with fewer misjudgments in their experimental setting.

The study is relevant to the authorization and security-monitoring aspects of our project because it illustrates how access decisions can take account of device behavior and changing risk. IBAC, RBAC, and ABAC provide different ways to define who may access an MQTT topic; risk-based control adds a further dimension by considering whether a device’s behavior remains trustworthy.

Our project does not currently implement this RBACE mechanism, so it should be described as a related approach or possible future extension, not as part of the implemented system.

Table 1: Article Table – Related Research Papers

Article Title	Year	Publisher	Author(s)	Relationship to the Project
Lightweight key Exchange for MQTT security	2026	Elsevier	.Singh et al	Lightweight key exchange & performance
Dynamic Access Control & End-to_End MQTT security	2026	Elsevier	Chen & Wilson	MQTT topic authorization & message
Systematic Review of Lightweight IOT Cryptography	2025	Springer Nature	Ansari & Ali	Background on ECC, encryption & integrity
MQTT-ZT Secure Broker	2025	IEEE	.James et al	ABAC & MQTT access control
Dynamic Resource-Aware Key Sharing for MQTT	2025	Elsevier	Kajanurmath & cholli	Lightweight Authentication & key sharing
Evaluation of MQTT Transport - & Application-Layer security	2026	Elsevier	Del-Valle-Soto et al	Compares TLS & ECC-AES cryptography & resource efficiency
Hybrid Cryptographic Framework for IOT security	2026	Springer Nature	Huan & Xie	Hybrid ECC-AES cryptography & resource efficiency
XMID-MQTT Intrusion detection	2025	Springer Nature	.Zejhida et al	Detects suspicious MQTT traffic using machine learning
MQTT Intrusion Detection Using Machine Learning	2025	IEEE	.Putro et al	Intrusion detection & evaluation metrics
Risk-Based access control for IOT	2025	Springer Nature	Ma & Chiu	Risk-based access decision for IOT devices

1.5 Practical Applications

The proposed IoT intrusion detection and secure communication system has several potential applications in environments where reliable data exchange, message integrity, and controlled access to connected devices are essential. By combining the MQTT communication protocol with HMAC-based message verification and access control mechanisms, the system is designed to improve the security of IoT communications while maintaining the efficiency required by resource-constrained devices.

1. Industrial Monitoring and Automation

In industrial environments, IoT sensors continuously transmit information about temperature, pressure, equipment status, and operating conditions. The proposed system can help protect MQTT messages against unauthorized modification and support access restrictions for devices and users. Access control can ensure that monitoring devices are permitted to publish sensor readings, while authorized control entities may access specific functions according to their assigned permissions.

2. Smart Agriculture and Irrigation Systems

Smart agriculture relies on sensors to monitor soil moisture, temperature, humidity, and other environmental conditions. These measurements may be transmitted through MQTT to a monitoring application or irrigation controller. Message integrity verification can help detect unauthorized modifications, while access control can restrict irrigation commands to authorized users or devices. This can improve the reliability and security of automated irrigation operations.

3. Smart Homes and Building Management

Smart homes and buildings use connected devices such as lighting systems, temperature sensors, door sensors, and environmental monitoring equipment. The proposed approach can support secure communication between these devices and their management applications. Access permissions can be assigned according to device identity, user role, or relevant attributes, helping prevent unauthorized entities from accessing restricted functions.

4. Environmental Monitoring

Environmental monitoring systems use distributed sensors to collect information about air quality, water conditions, temperature, and humidity. These systems require reliable transmission of measurements to centralized monitoring platforms. HMAC-based verification can help identify messages that have been altered or generated without knowledge of the appropriate secret key, while access control can restrict which entities may publish or retrieve specific data.

5. Smart Energy Management

Smart energy systems use connected meters and sensors to monitor electricity consumption and equipment performance. Unauthorized changes to transmitted measurements or control commands may affect monitoring accuracy and operational decisions. The proposed system can provide message integrity verification and access restrictions to help protect communication between energy-monitoring devices and authorized management applications.

6. Role-Based and Attribute-Based IoT Access Management

IoT environments may contain many users and devices with different responsibilities and access requirements. Integrating Identity-Based Access Control (IBAC), Role-Based Access Control (RBAC), and Attribute-Based Access Control (ABAC) provides a basis for examining different approaches to authorization. These models can be applied to MQTT publishing and subscription

operations, allowing access decisions to be based on identity, assigned role, or relevant attributes. Their effectiveness can then be evaluated through controlled experiments.

Overall, these applications demonstrate the potential value of combining message integrity verification, intrusion detection, and access control within an MQTT-based IoT security architecture. The suitability of the proposed system for each application depends on its implementation, configuration, and experimental evaluation.

1.6 Research Challenges

Despite the benefits of using HMAC technology to enhance the security of Internet of Things (IoT) communications, several challenges must be considered when developing and evaluating the proposed system.

1. Key Management

HMAC relies on a shared secret key to generate and verify message authentication codes. Therefore, securely generating, distributing, storing, and updating these keys is a major challenge. If an unauthorized person obtains the secret key, they may be able to generate valid authentication codes and impersonate a trusted device.

2. Replay Attacks

HMAC helps verify message integrity and authenticity, but it does not automatically prevent replay attacks. An attacker may capture a valid message and attempt to send it again. Additional mechanisms, such as timestamps, sequence numbers, or nonces, may be required to detect repeated or outdated messages.

3. Resource Constraints

Many IoT devices have limited processing power, memory, and battery capacity. Although HMAC is generally efficient, generating and verifying authentication codes still requires computational resources. The system must therefore balance security requirements with the limited capabilities of IoT devices.

4. Access Control Management

Authenticating a message does not necessarily mean that the sender is authorized to perform the requested action. The system must distinguish between verifying a device's identity and deciding which resources or operations that device may access. Implementing and comparing IBAC, RBAC, and ABAC introduces additional design and management considerations.

5. MQTT Broker Security

MQTT provides a lightweight communication mechanism for IoT devices, but protecting the broker and controlling access to topics remain important challenges. The system must ensure that devices can publish or subscribe only to the topics and operations permitted by the applicable security policies.

6. Scalability

As the number of connected IoT devices increases, the system must handle more messages, authentication operations, keys, and access requests. Maintaining efficient message verification and access control as the system grows can become challenging.

7. Security and Performance Trade-Off

Increasing security may introduce additional processing time and communication overhead. The system must balance effective authentication and access control with acceptable message-processing delays and resource consumption.

Table 2: IBAC, RBAC and ABAC comparison

MODEL	EXPLANATION	ADVANTAGES	DISADVANTAGE	APPLICATION TO OUR PROJECT
RBAC	Permissions are assigned according to user or device roles.	Simple to manage; easy to understand; suitable for systems with clearly defined roles.	Less flexible permissions depend on device conditions or context; role management may become complex as the system grows.	Assign sensor to the publisher and Monitor to the subscriber. Allow the Sensor to publish to iot/message and the Monitor to subscribe to it
ABAC	Access decisions depend on attributes of the device, resource, action, and possibly the environment.	Flexible and fine-grained; supports context-aware policies; scales well when many device attributes are involved.	More complex to design, configure, and evaluate; requires reliable attribute information and policy management.	Allow publishing only if the device is registered, its type is Sensor, its status is approved, and the requested topic is iot/message.
IBAC	Permissions are assigned directly to specific registered device identities.	Direct and precise permissions; easy to understand in small systems; supports device-specific restrictions.	Administrative overhead increases with the number of devices; changing permissions for many devices can be cumbersome.	Register device_01 as an authorized publisher and device_02 as an authorized subscriber. Deny publishing or subscribing by unknown or unauthorized identities.

Chapter 1:

1.1 Introduction to the Internet of Things (IoT)

The Internet of Things (IoT) is a technological concept that enables physical devices to communicate and exchange data through network connections. These devices may include sensors, actuators, embedded systems, smart appliances, and industrial equipment. IoT systems collect information from their surrounding environments and transmit it to other devices or software applications for monitoring, analysis, and automated decision-making.

IoT technology is used in several fields, including smart homes, healthcare monitoring, industrial automation, agriculture, transportation, and smart cities. For example, a temperature sensor can periodically publish measurements to a monitoring application, while an industrial sensor can transmit operational data to a central monitoring system.

Despite these advantages, IoT environments introduce important security challenges. Many IoT devices have limited processing power, memory, storage, and energy resources. Furthermore, they may operate in environments where physical access, network exposure, and device management are difficult to control. These conditions make it important to implement security mechanisms that protect communications without imposing excessive computational overhead [8].

In this project, the theoretical study focuses on securing communications between IoT applications using the Message Queuing Telemetry Transport (MQTT) protocol. Particular attention is given to message integrity, sender authentication, confidentiality, and access control.

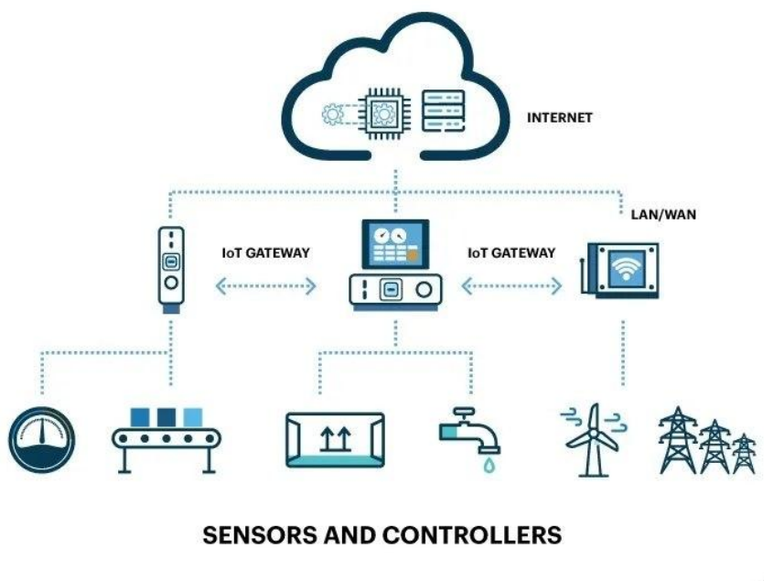


Figure 1.1: Overview of the Internet of Things (IoT)

1.2 IoT Architecture

IoT architecture describes the main components involved in collecting, transmitting, processing, and using data generated by connected devices. Although different architectural models exist, a commonly used conceptual model consists of three layers: the perception layer, the network layer, and the application layer.

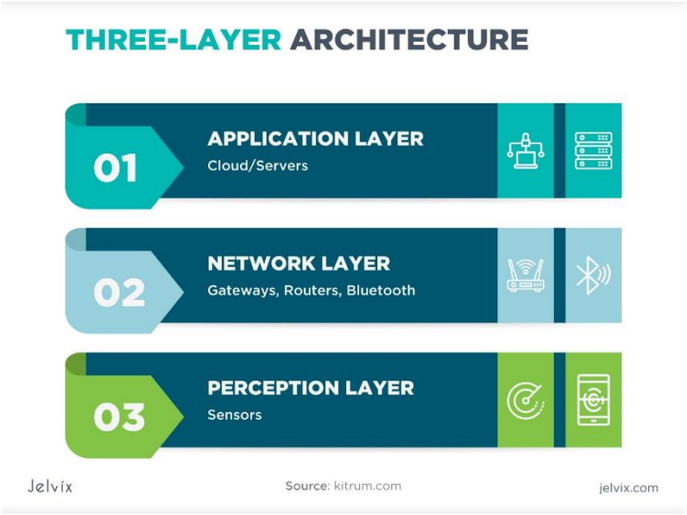


Figure 1.2: Three-Layer Architecture of the Internet of Things

1.2.1 Perception Layer

The perception layer contains physical devices responsible for sensing environmental conditions and interacting with the physical world. Examples include temperature sensors, motion detectors, cameras, and actuators.

These devices generate the information used by IoT applications. However, their limited computational resources and exposure to physical environments can make them vulnerable to unauthorized access, device manipulation, and malicious data injection.

1.2.2 Network Layer

The network layer transfers information between IoT devices, gateways, brokers, servers, and applications. It may use technologies such as Wi-Fi, Ethernet, cellular networks, and other communication technologies.

This layer is essential for transporting messages, but communication channels can be exposed to threats such as interception, unauthorized connections, message modification, and denial-of-service attacks. Appropriate communication security mechanisms are therefore required to protect data in transit.

1.2.3 Application Layer

The application layer provides services that use IoT data. These services may include monitoring dashboards, automated control systems, alerting applications, and data-analysis platforms.

At this layer, security mechanisms can determine whether a received message is authentic, whether its content has been modified, and whether a particular client is authorized to access a resource or perform an operation.

For the proposed project, the application layer is particularly important because message authentication and authorization can be implemented within the applications that publish and receive MQTT messages.

The three-layer model provides a simplified view of IoT architecture. Actual deployments may use additional layers, gateways, cloud services, and device-management components [8].

1.3 MQTT Protocol

Message Queuing Telemetry Transport (MQTT) is a lightweight messaging protocol designed for communication between applications and devices over network connections. It is particularly useful in environments where bandwidth, processing resources, and communication reliability are limited.

MQTT uses a publish/subscribe communication model rather than requiring every sender to communicate directly with every receiver. In this model, clients publish messages to topics, while other clients subscribe to the topics whose messages they want to receive. An MQTT broker manages message distribution between publishers and subscribers [9].

For example, a temperature-monitoring device may publish a measurement to the topic "iot/temperature". A monitoring application subscribed to that topic can then receive the published measurement through the broker.

MQTT supports different Quality of Service (QoS) levels that determine the delivery guarantees associated with message transmission. These levels provide different trade-offs between delivery reliability and communication overhead.

Although MQTT is suitable for resource-constrained environments, using MQTT alone does not automatically guarantee that the application-level contents of every message are authentic or have remained unchanged. A deployment must select and configure suitable security controls, including connection protection, client authentication, authorization, and, where required, application-level message verification [9].

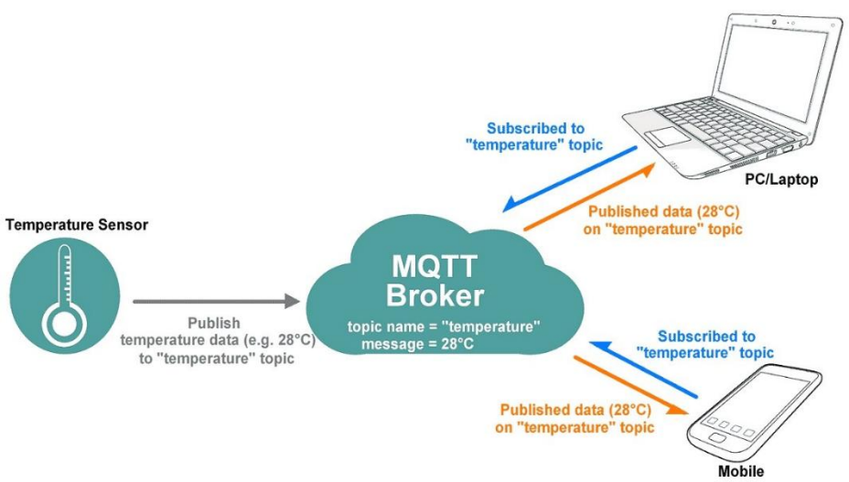


Figure 1.3: MQTT Protocol in IoT Communication

1.4 MQTT Architecture and Communication Model

The MQTT communication model primarily consists of publishers, subscribers, a broker, and topics.

1.4.1 Publisher

A publisher is a client that sends a message to the MQTT broker. It may be a sensor, an embedded device, or a software application.

A publisher selects the topic associated with its message and transmits the message to the broker. Depending on the system design, the application may also attach security-related information, such as a message authentication code, to support verification by the receiving application.

1.4.2 Subscriber

A subscriber is a client that receives messages published to topics to which it has subscribed.

In a security-enhanced application, a subscriber should not automatically trust every received payload. It can first verify the message authentication code and then process the message only if verification succeeds. If the application also uses encryption, the message must be decrypted according to the defined protocol and key-management process.

1.4.3 MQTT Broker

The broker is the central component responsible for receiving published messages and distributing them to matching subscribers. It also manages client connections and subscriptions.

Broker-level security can restrict which clients may connect, publish to particular topics, or subscribe to particular topics. However, broker permissions and application-level message verification address different security requirements. Broker authorization controls permitted MQTT operations, while an application-level HMAC can help a receiver verify message integrity and possession of a shared secret.

1.4.4 Topics and Message Delivery

An MQTT topic identifies the category or destination of a message. Topics allow publishers and subscribers to communicate without needing direct knowledge of each other.

For example, a system may use "iot/message" as the topic for exchanging test messages between two simulated IoT applications. The publisher sends data through the broker, and the subscriber receives messages associated with that topic.

The separation between publishers and subscribers simplifies system design. Nevertheless, secure operation requires more than successful message delivery: the system must also consider the origin, integrity, confidentiality, and authorization of the exchanged data [9].

1.5 IoT Security Challenges

IoT systems face several security challenges because they combine large numbers of connected devices, diverse communication technologies, and potentially limited device resources.

1.5.1 Data Integrity

Data integrity means that information remains accurate and has not been modified in an unauthorized manner during storage or transmission. If an attacker changes an IoT message, the receiving application may make incorrect decisions based on the altered information.

A message authentication mechanism, such as HMAC, can help detect unauthorized changes to authenticated message contents when the secret key remains protected.

1.5.2 Authentication

Authentication is the process of verifying the identity of a user, device, client, or communicating application. Without suitable authentication, an unauthorized client may attempt to impersonate a legitimate device or send misleading data.

MQTT deployments can authenticate clients at the broker, while applications may use additional mechanisms to verify the origin of individual messages. These mechanisms serve related but distinct purposes.

1.5.3 Confidentiality

Confidentiality protects information from being disclosed to unauthorized parties. IoT messages may contain sensitive measurements, operational information, or personal data.

HMAC does not encrypt message contents. Confidentiality requires a separate mechanism, such as properly implemented encryption or a secure transport channel. The Advanced Encryption Standard (AES), for example, is a symmetric encryption algorithm that can be used as part of a suitable data-protection design.

1.5.4 Authorization

Authorization determines which authenticated entities are permitted to access resources or perform specific operations. For example, a monitoring client may be allowed to subscribe to sensor measurements but not to publish commands that control an actuator.

Without appropriate authorization, a validly connected client may still perform operations beyond its intended permissions. Access-control rules should therefore be designed according to the roles, identities, attributes, and resources involved in the system.

1.5.5 Resource Constraints

Many IoT devices have limited CPU capacity, memory, battery power, and network bandwidth. Security mechanisms that introduce substantial processing or communication overhead may reduce responsiveness or increase energy consumption.

Consequently, IoT security design should balance the required protection with the capabilities and operational requirements of the devices. Lightweight mechanisms can be useful, but their suitability must be assessed in the context of the complete system [8].

1.5.6 Availability

Availability means that authorized users and devices can access system services when required. Network congestion, broker failures, excessive connection attempts, and denial-of-service attacks can interrupt communication.

Security controls should therefore be designed to protect the system without creating unnecessary processing costs or single points of failure.

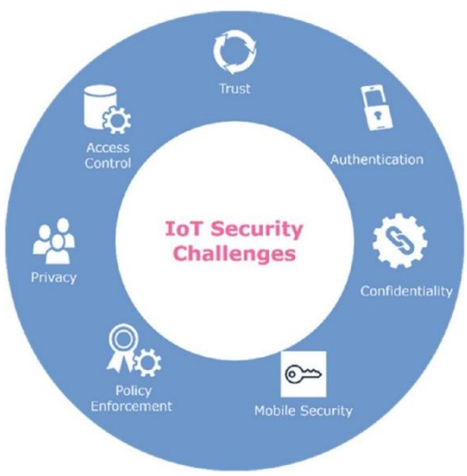


Figure 1.4: Common Security Challenges in IoT Systems

1.6 Common Attacks on MQTT-Based IoT Systems

MQTT-based IoT systems may be exposed to several categories of attacks, depending on the network configuration, broker permissions, client authentication, and application design.

1.6.1 Message Tampering

Message tampering occurs when an unauthorized party modifies data during communication. For example, a transmitted sensor value may be changed before it reaches the receiving application.

If the receiver does not verify message integrity, the altered value may be accepted as legitimate. HMAC-based verification can help detect changes to the authenticated content, provided that the receiver uses the correct secret key and verifies the correct message representation [10].

1.6.2 Client Impersonation

Client impersonation occurs when an attacker attempts to act as a legitimate device or application. Weak credentials, inadequate client authentication, or poorly managed shared secrets may increase this risk.

Broker-level authentication helps determine whether a client is permitted to establish a connection. An application-level HMAC can provide additional evidence that a message was generated by an entity possessing the relevant shared secret. However, HMAC does not independently establish a real-world identity or prevent every form of impersonation.

1.6.3 Unauthorized Publishing and Subscription

An improperly configured MQTT broker may allow a client to publish to sensitive topics or subscribe to messages that should be restricted.

This threat is primarily addressed through authorization policies that define which clients may publish or subscribe to particular topics. Application-level verification can complement these policies but should not replace broker access controls.

1.6.4 Message Replay

A replay attack occurs when a previously valid message is transmitted again in an attempt to make the receiver accept old information as new.

HMAC alone does not prevent replay. A previously valid message and its authentication code may remain valid if the receiver has no mechanism to determine whether the message has already been processed.

Replay protection generally requires additional protocol information and verification, such as sequence numbers, timestamps, nonces, or unique message identifiers, together with suitable rules for rejecting duplicates or outdated messages.

1.6.5 Denial of Service

A denial-of-service (DoS) attack attempts to make a service unavailable by exhausting its resources or disrupting communication. An MQTT broker or receiving application may be affected by excessive connections, large numbers of messages, or repeated requests.

Possible defensive measures include connection limits, rate limiting, resource monitoring, appropriate broker configuration, and network-level protections. Cryptographic verification alone does not prevent denial-of-service attacks.

These attack categories demonstrate why a secure MQTT system requires multiple complementary controls rather than relying on a single cryptographic mechanism [9], [10].

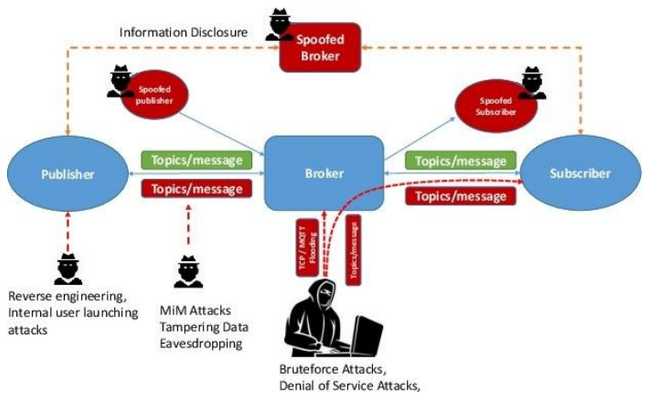


Figure 1.5: Common Security Threats in MQTT-Based IoT Systems

1.7 Cryptography in IoT

Cryptography provides mechanisms for protecting information and verifying communication between devices and applications. The selection of a cryptographic mechanism depends on the required security properties, device capabilities, and system architecture.

1.7.1 Symmetric Encryption

Symmetric encryption uses a secret key for encryption and decryption. The Advanced Encryption Standard (AES) is a widely used symmetric encryption algorithm.

When implemented using an appropriate mode and secure key-management process, symmetric encryption can protect the confidentiality of IoT message contents. However, encryption by itself does not necessarily provide integrity or authenticate the sender. These properties depend on the selected construction and protocol.

1.7.2 Asymmetric Cryptography

Asymmetric cryptography uses related public and private keys. Depending on the algorithm and protocol, it can support digital signatures, authentication, and secure key establishment.

Elliptic Curve Cryptography (ECC) is an approach based on elliptic-curve mathematical structures. Suitable elliptic-curve protocols can offer security with comparatively small key sizes, although their computational cost and implementation requirements still need to be considered for constrained devices.

1.7.3 Hash Functions

A cryptographic hash function transforms input data of arbitrary length into a fixed-length digest. Hash functions are used in many security applications, including integrity-related constructions and digital signatures.

A plain hash does not authenticate a message because anyone who can modify the message can generally calculate a new hash for the modified content. Message authentication requires an additional mechanism, such as a secret-key-based construction.

1.7.4 Message Authentication

Message authentication mechanisms help a receiver verify that a message is consistent with an authentication value generated using the required key or cryptographic process. HMAC is one such mechanism and is particularly relevant when the communicating applications share a secret key.

In a layered security design, encryption may protect confidentiality, HMAC may provide message integrity and key-based authentication, and access-control policies may restrict the operations available to each client. These mechanisms address different security requirements and should not be treated as interchangeable [10].

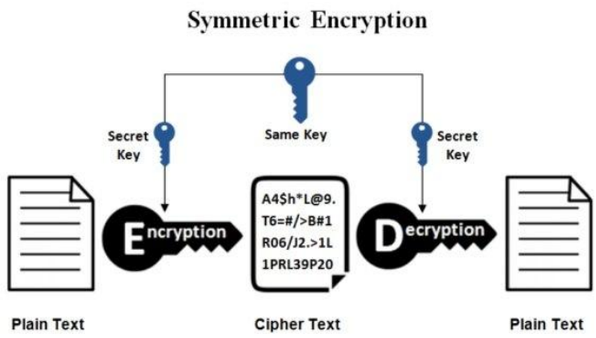


Figure 1.6: Role of Cryptography in IoT Security

1.8 Hash-Based Message Authentication Code (HMAC)

Hash-Based Message Authentication Code (HMAC) is a cryptographic construction that combines a secret key with a cryptographic hash function to produce an authentication code for a message. It is defined in RFC 2104 and can be used to verify message integrity and demonstrate possession of a shared secret key [10].

HMAC is relevant to IoT communication because it can be implemented at the application layer and does not require the message itself to be encrypted as part of the HMAC operation. The choice of hash function, key length, message format, and key-management procedure must follow the security requirements of the system.

1.8.1 HMAC Generation

The sender and receiver must possess the appropriate shared secret key. When the sender prepares a message, it calculates an HMAC over the data that the protocol defines as authenticated. The sender then transmits the message together with the resulting authentication code.

The receiver uses its copy of the shared secret key to calculate an expected authentication code over the received authenticated data. It compares this value with the authentication code included in the message using an appropriate verification function.

If verification succeeds, the receiver has evidence that the authenticated data has not been modified without detection and that the message was produced by an entity possessing the shared secret, subject to the security of the algorithm, key, and implementation.

If verification fails, the receiver should reject the message rather than process it as an authenticated message.

1.8.2 Message Integrity

HMAC allows the receiver to detect unauthorized changes to the authenticated message contents. If an attacker modifies those contents without knowing the secret key, the attacker should not be able to generate a valid authentication code for the modified message, assuming a secure hash function and correct implementation.

The exact fields included in the HMAC calculation are important. If security-relevant fields are omitted from the authenticated data, those fields may be modified without invalidating the authentication code.

1.8.3 Key-Based Authentication

HMAC also provides a form of message authentication based on possession of a shared secret key. A successful verification indicates that the message is consistent with an authentication code generated using that key.

However, HMAC does not independently identify a particular individual or device when multiple entities share the same secret. It also does not provide non-repudiation, because the same secret may be used to generate valid authentication codes by any party that possesses it.

1.8.4 Limitations of HMAC

Although HMAC provides important security properties, it does not encrypt the message or directly provide authorization. It also does not automatically prevent replay attacks.

The secret key must be generated, distributed, stored, and protected securely. If an unauthorized party obtains the key, that party may be able to generate valid authentication codes. The system must also ensure that the sender and receiver authenticate exactly the same data representation and reject invalid messages consistently.

Therefore, HMAC should be considered one component of a broader security design that includes appropriate encryption, key management, authentication, authorization, and replay protection where required [10].

1.9 Why HMAC Is Suitable for MQTT-Based IoT Systems

HMAC can be suitable for MQTT-based IoT applications because it provides message integrity and shared-secret-based authentication without requiring the message contents to be encrypted as part of the HMAC operation.

First, HMAC can be applied at the application layer. A publisher can calculate an authentication code before publishing a message, and a subscriber can verify the code after receiving the message through the MQTT broker. This allows message verification to be incorporated into the application without changing the fundamental MQTT publish/subscribe communication model.

Second, HMAC uses a secret key and a cryptographic hash function. Its computational requirements depend on the chosen hash function, message size, and implementation. These factors should be evaluated for the intended IoT environment rather than assuming that every implementation will have negligible overhead.

Third, HMAC supports detection of unauthorized modifications to the authenticated message contents. This is valuable when receiving applications must reject data that cannot be successfully verified.

Finally, HMAC can complement other security mechanisms. Encryption can be used when confidentiality is required, broker authentication can control client connections, and authorization policies can restrict access to MQTT topics and operations.

However, HMAC alone does not guarantee complete MQTT security. It does not conceal message contents, independently assign permissions, or automatically prevent replay. A secure implementation must account for these requirements separately and protect the shared secret key [9], [10].

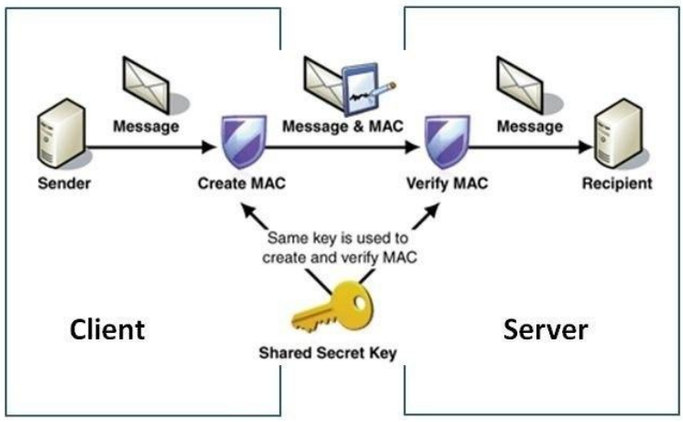


Figure 1.7: HMAC-Based Message Authentication Process

1.10 Authorization and Access Control Models

Authorization is a security process that determines which actions an authenticated user, device, or application is permitted to perform. In an IoT system, authorization may control access to MQTT topics, message publishing, subscriptions, configuration functions, and device-control operations.

Authentication and authorization serve different purposes. Authentication attempts to establish the identity or credentials of an entity, whereas authorization evaluates whether that entity is allowed to perform a requested operation. A client may authenticate successfully but still be denied access to a particular topic.

For example, an IoT monitoring application may be permitted to subscribe to sensor measurements, while a control application may be permitted to publish commands to an actuator. These permissions should be defined explicitly and enforced by the appropriate system components.

Access-control models provide structured ways to express and enforce these permissions. This project considers three approaches: Identity-Based Access Control (IBAC), Role-Based Access Control (RBAC), and Attribute-Based Access Control (ABAC). Because the term IBAC is not used with one universally standardized meaning across all access-control literature, its precise definition should be stated clearly in the project methodology.

1.10.1 Identity-Based Access Control (IBAC)

In this project, Identity-Based Access Control (IBAC) refers to an approach in which permissions are assigned directly to identified users, devices, or clients. An access request is evaluated by checking the requesting identity against the permissions associated with that identity.

For example, a client identified as "sensor_01" may be permitted to publish to "iot/temperature", while another client may be permitted only to subscribe to that topic.

The principal advantage of identity-based control is that permissions can be defined specifically for each identity. This may be appropriate in small systems with a limited number of clients and relatively simple access requirements.

However, managing individual permissions may become more complex as the number of devices increases. Adding or replacing devices may require changes to multiple access-control entries. The

model's exact capabilities depend on how identities and permission rules are represented in the implementation.

For a fair comparison, the project should specify which identity information is checked, how permissions are stored, and which operations are allowed or denied.

1.10.2 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) assigns permissions to roles rather than defining every permission independently for every user or device. Users or devices are then associated with the roles appropriate to their responsibilities.

For example, an IoT system may define the following roles:

- Sensor: permitted to publish measurements to assigned topics.
- Monitor: permitted to subscribe to selected measurement topics.
- Administrator: permitted to manage system settings and access policies.

When a client requests an operation, the system determines its assigned role and checks whether that role includes the required permission.

RBAC can simplify permission management because multiple clients can share a common set of permissions through their roles. Changes to a role's permissions can be applied consistently to all clients assigned to that role.

However, RBAC requires suitable role definitions and assignment rules. If the system needs to make decisions based on changing contextual information, such as device location or time, additional rules or mechanisms may be necessary. The formal RBAC model and its variations are discussed in established access-control literature [12].

1.10.3 Attribute-Based Access Control (ABAC)

Attribute-Based Access Control (ABAC) determines whether an operation is allowed by evaluating attributes associated with the requesting entity, the target resource, the requested action, and, where relevant, the surrounding environment.

Attributes may include device identity, device type, topic name, requested operation, time, location, or a defined security classification. An authorization policy evaluates relevant attributes to reach an access decision.

For example, a policy could permit a sensor to publish measurements only to its assigned topic, while allowing a monitoring application to subscribe to that topic. A further policy could restrict an administrative operation to authorized clients under specified conditions.

ABAC can express more context-sensitive rules than a simple list of individual permissions or a basic role assignment. Nevertheless, its effectiveness depends on accurate attributes, clearly defined policies, reliable evaluation, and careful testing.

The NIST definition of ABAC describes authorization decisions based on attributes of subjects, objects, operations, and, where applicable, environmental conditions [11].

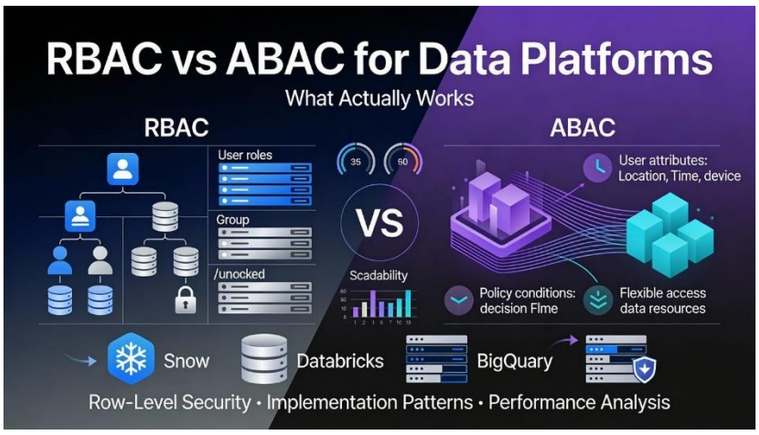


Figure 1.8: Overview of Access Control Models

1.10.4 Applying Access Control to MQTT Communication

In an MQTT-based IoT system, access control can be applied to operations such as connecting to the broker, publishing messages, and subscribing to topics.

A policy should specify which clients are permitted to publish to each topic and which clients may receive messages from it. For example, a sensor may be allowed to publish measurements but not to publish administrative commands. A monitoring client may be allowed to subscribe to measurements without receiving permission to control devices.

The broker may enforce connection and topic permissions through its supported authentication and authorization configuration. An application may also implement additional checks when processing messages.

HMAC and access control should remain separate concepts. HMAC verification helps establish the integrity of authenticated message contents and evidence of possession of a shared secret. Authorization determines whether the requesting identity or client is permitted to perform an operation. A message with a valid HMAC should not automatically be accepted for every operation if the sender lacks the required permission.

When comparing IBAC, RBAC, and ABAC, the same basic communication scenarios should be used wherever possible. The evaluation should record whether each request is allowed or denied according to the defined policy, and should distinguish correct authorization decisions from message-integrity verification.

References

[1] OASIS, *MQTT Version 5.0*, OASIS Standard, 2019.

[2] *Regulating Data Sharing Across MQTT Environments*, Journal of Network and Computer Applications, vol. 174, article 102907, 2021.

[3] NIST, *Advanced Encryption Standard (AES)*, FIPS 197.

[4] H. Krawczyk, M. Bellare, and R. Canetti, *HMAC: Keyed-Hashing for Message Authentication*, RFC 2104, 1997.

[5] IETF, *Fundamental Elliptic Curve Cryptography Algorithms*, RFC 6090, 2011.

[6] V. C. Hu et al., *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*, NIST Special Publication 800-162, 2014, updated 2019.

[7] R. Sandhu, D. Ferraiolo, and R. Kuhn, *The NIST Model for Role-Based Access Control: Towards a Unified Standard*, 2000.

[8] M. Fagan, K. N. Megas, K. Scarfone, and M. Smith, IoT Device Cybersecurity Capability Core Baseline. NIST Interagency/Internal Report 8259A, National Institute of Standards and Technology, May 2020. https://doi.org/10.6028/NIST.IR.8259A

[9] A. Banks, E. Briggs, K. Borgendale, and R. Gupta, eds., MQTT Version 5.0. OASIS Standard, March 7, 2019.

[10] H. Krawczyk, M. Bellare, and R. Canetti, “HMAC: Keyed-Hashing for Message Authentication,” RFC 2104, Internet Engineering Task Force (IETF), February 1997.

[11] V. C. Hu, D. Ferraiolo, R. Kuhn, et al., Guide to Attribute Based Access Control (ABAC) Definition and Considerations. NIST Special Publication 800-162, National Institute of Standards and Technology, updated August 2019.

[12] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli, Role-Based Access Control, 2nd ed. Artech House, 2007.